

Assignment 3

```
library(tidyverse)
library(nnet)
library(survival)
library(survminer)
```

Question 2

(a) Read in the data and display at least some of your data frame. Do you have what you expected?

Given that these values are separated by a single space, we can use `read_delim` with a space delimiter

```
url = "https://www.uts.utoronto.ca/~butler/d29/mercury.txt"
merc = read_delim(url, " ")
```

```
## Parsed with column specification:
## cols(
##   mercury = col_double(),
##   lake = col_double()
## )
```

And let's ensure that our data looks okay

```
head(merc)
```

```
## # A tibble: 6 x 2
##   mercury lake
##   <dbl> <dbl>
## 1    1.05     3
## 2    0.23     3
## 3    0.1      2
## 4    0.77     3
## 5    0.91     3
## 6    0.25     3
```

It looks as expected

(b) Why is this not the kind of model you would fit with `polr`? Explain briefly

It is only appropriate to use `polr` when you would want an ordinal regression. That is, it matters when the response variable can be ordered in some way (e.g. greatest to least).

Since we don't have that much knowledge of the lake's labelling procedure (as it is deemed insignificant), it would be ideal to fit a nominal regression using `multinom`

(c) Fit a suitable model for predicting lake type from mercury content

As stated in (b), we will use `multinom`

```
merc.1 = multinom(factor(lake)~mercury, data=merc)
```

```
## # weights:  9 (4 variable)
## initial  value 38.451430
## iter   10 value 28.884248
## final   value 28.859596
## converged
```

and we can use summary

```
summary(merc.1)
```

```
## Call:
## multinom(formula = factor(lake) ~ mercury, data = merc)
##
## Coefficients:
## (Intercept)  mercury
## 2    -1.803544 10.20089
## 3    -2.747468 11.67057
##
## Std. Errors:
## (Intercept)  mercury
## 2     1.619714 5.985448
## 3     1.682489 6.030354
##
## Residual Deviance: 57.71919
## AIC: 65.71919
```

(d) Test whether mercury content is significant in predicting lake type. What do you conclude?

Using drop1 on a multinom object does not work.

So we take out mercury, fit again and compare using anova:

```
maine.2 = update(merc.1, .~.-mercury)
```

```
## # weights:  6 (2 variable)
## initial  value 38.451430
## final   value 33.909833
## converged
```

then we conduct an anova:

```
anova(maine.2, merc.1)
```

##	Model	Resid. df	Resid. Dev	Test	Df	LR stat.	Pr(Chi)
## 1	1	68	67.81967		NA	NA	NA
## 2	mercury	66	57.71919	1 vs 2	2	10.10047	0.006407813

The probability is low, so we should keep mercury. That is, it has a significant effect in predicting lake type.

Do not use step here.

(e) For mercury values 0.1, 0.2, 0.5, 1, 1.5, predict lake type probabilities for each of them, and display your results side by side with the values they are predictions for

First, let's create a data frame

```
small = data.frame(mercury=c(0.1, 0.2, 0.5, 1, 1.5))
small
```

```
##   mercury
## 1      0.1
## 2      0.2
## 3      0.5
## 4      1.0
## 5      1.5
```

Then let's predict

```
p = predict(merc.1, small, type="probs")
cbind(small, p)
```

```
##   mercury          1          2          3
## 1      0.1 6.014259e-01 0.2747463 0.1238278
## 2      0.2 3.414829e-01 0.4326510 0.2258661
## 3      0.5 2.001687e-02 0.5410312 0.4389519
## 4      1.0 8.375501e-05 0.3714776 0.6284387
## 5      1.5 3.034763e-07 0.2208728 0.7791269
```

A bit hard to see, so let's just round it

```
compare = cbind(small, round(p, 2))
compare
```

```
##   mercury    1    2    3
## 1      0.1 0.60 0.27 0.12
## 2      0.2 0.34 0.43 0.23
## 3      0.5 0.02 0.54 0.44
## 4      1.0 0.00 0.37 0.63
## 5      1.5 0.00 0.22 0.78
```

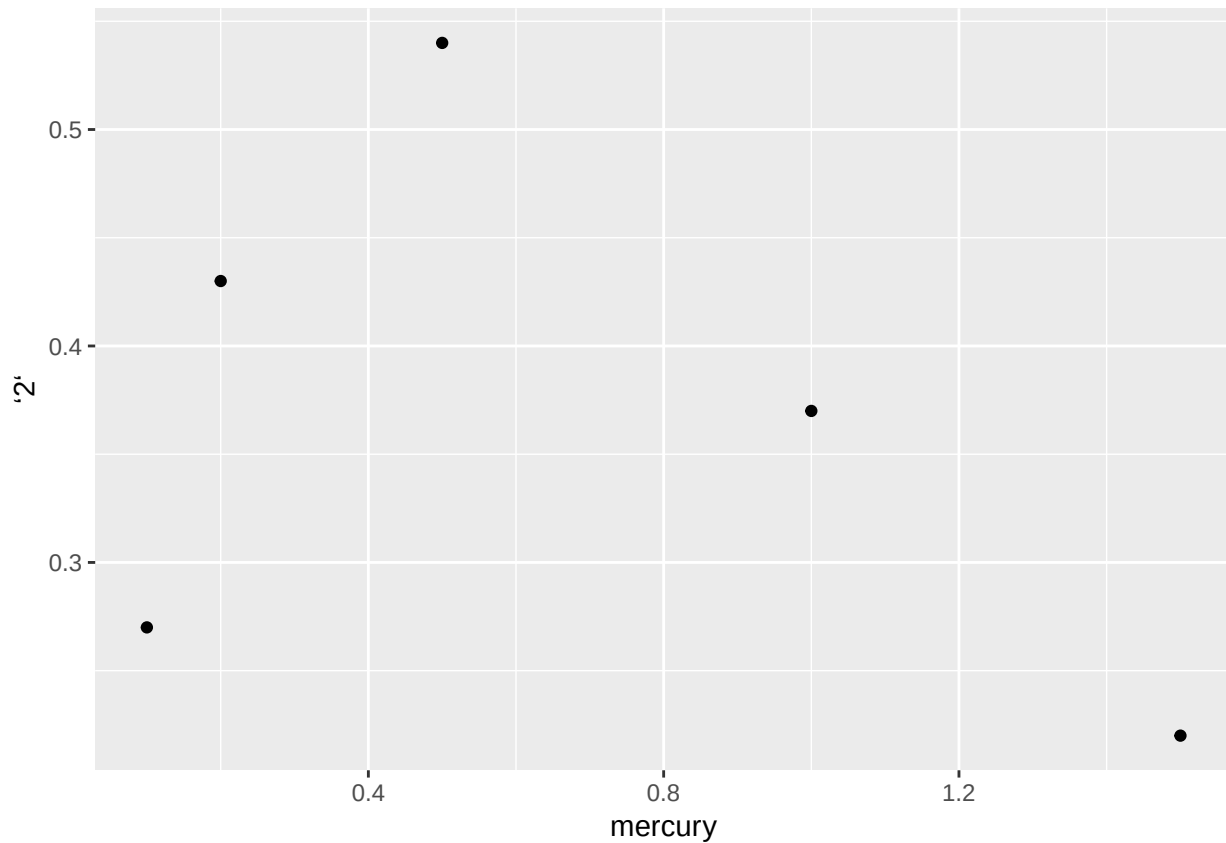
Much better.

(f) Describe how predicted lake type changes as mercury content increases.

If we look at a small mercury value, say 0.1 or 0.2, we see that there are existant probabilities in lake 1. In this case, we can somewhat elude to the idea that if the mercury content is small, it most likely would be coming from lake 1 (especially when lake 1 has the highest probability with lowest mercury listed at 0.1). However, if mercury content increases, the likeliness of the water coming from lake 1 is non existant.

As for lake 2, it appears that there is a “sweet spot” of probability for mercury content:

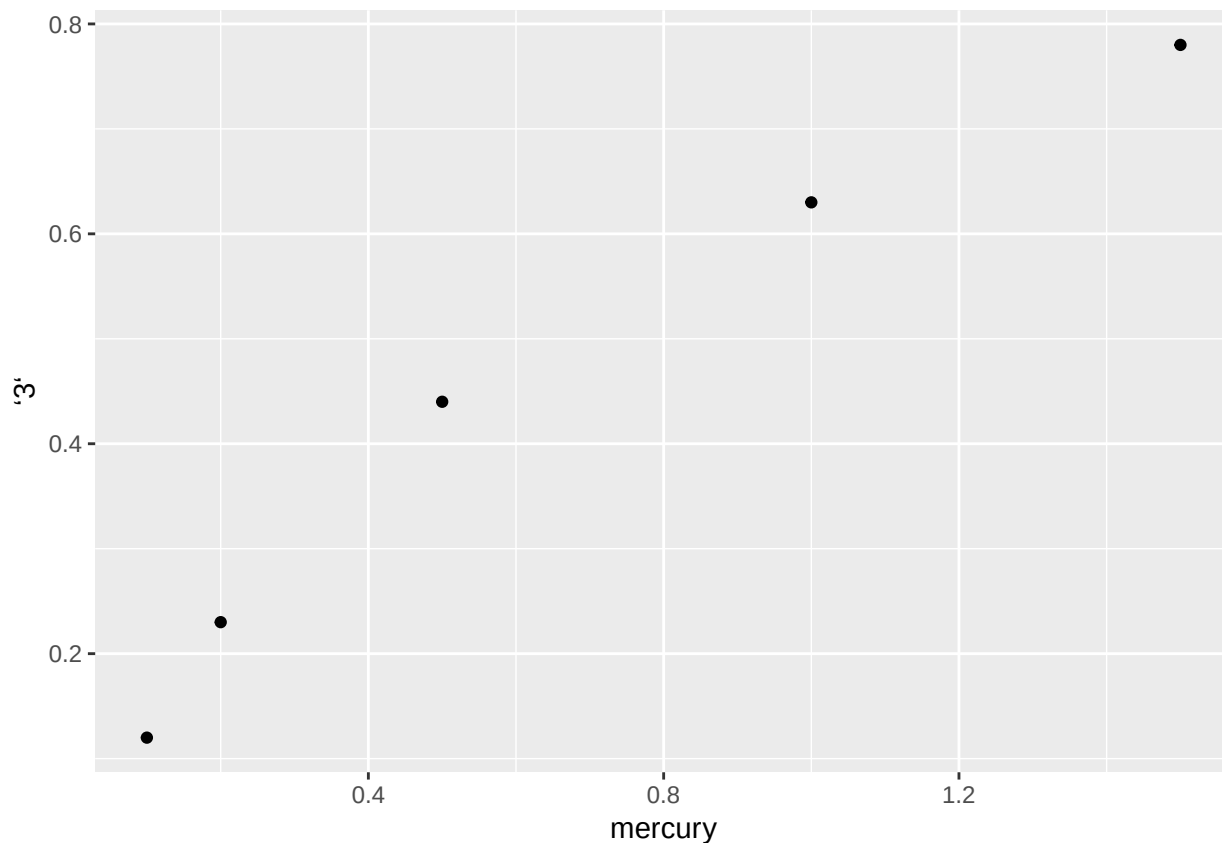
```
ggplot(compare, aes(x=mercury, y=`2`)) + geom_point() # backquotes is necessary!
```



It looks like it is curving. At the very least, in the range from $[0, 0.5]$, as mercury content increases there is a higher probability that the water may be coming from lake 2. As mercury content increases from 1.0 and beyond, the likeliness of water coming from lake 2 decreases.

As for lake 3, we can simply come to the conclusion that the higher the mercury content, the higher the probability that it may come from lake 3:

```
ggplot(compare, aes(x=mercury, y=`3`)) + geom_point() # backquotes is necessary!
```



Question 4

(a) What are the full names of the three treatments (as shown in the help for the data set)? The data set only contains abbreviations of these names

Let me cheat a bit by looking at the man page of colon

```
?colon
```

```
## Help on topic 'colon' was found in the following packages:
```

```
##
```

```
##   Package          Library
##   base             /opt/R/3.5.2/lib/R/library
##   survival         /opt/R/3.5.2/lib/R/library
```

```
##
```

```
##
```

```
## Using the first match ...
```

Appears that rx indicates the type of treatment; and let's just list them out:

```
colon %>% distinct(rx)
```

```
##      rx
```

```
## 1 Lev+5FU
```

```
## 2   Obs
```

```
## 3   Lev
```

- Obs stands for observation
- Lev stands for Levamisole and
- Lev+5FU also stands for Levamisole+5-FU (however, after googling a bit, it appears that it stands for levamisole and flurouacil).

(b) Read in and display some of the data. How many subjects do you have?

I will be trusting Professor Butler's csv file:

```
url = "https://www.utoronto.ca/~butler/d29/colon.csv"
tidy.colon = read_csv(url)
```

```
## Parsed with column specification:
## cols(
##   id = col_double(),
##   rx = col_character(),
##   sex = col_character(),
##   age = col_double(),
##   obstruct = col_character(),
##   perfor = col_character(),
##   adhere = col_character(),
##   nodes = col_double(),
##   status = col_character(),
##   differ = col_character(),
##   extent = col_character(),
##   surg = col_character(),
##   node4 = col_character(),
##   time = col_double()
## )
```

I think glimpse is a good indicator of this.

```
glimpse(tidy.colon)
```

```
## Observations: 888
## Variables: 14
## $ id      <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16...
## $ rx      <chr> "Lev+5FU", "Lev+5FU", "Obs", "Lev+5FU", "Obs", "Lev+5...
## $ sex     <chr> "male", "male", "female", "female", "male", "female",...
## $ age     <dbl> 43, 63, 71, 66, 69, 57, 77, 54, 46, 68, 47, 52, 64, 6...
## $ obstruct <chr> "no", "no", "no", "yes", "no", "no", "no", "no", "no"...
## $ perfor  <chr> "no", "no", "no", "no", "no", "no", "no", "no", "no",...
## $ adhere  <chr> "no", "no", "yes", "no", "no", "no", "no", "no", "yes...
## $ nodes   <dbl> 5, 1, 7, 6, 22, 9, 5, 1, 2, 1, 1, 2, 1, 3, 4, 1, 6, 1...
## $ status  <chr> "recur", "no recur", "recur", "recur", "recur", "recu...
## $ differ  <chr> "moderate", "moderate", "moderate", "moderate", "mode...
## $ extent  <chr> "serosa", "serosa", "muscle", "serosa", "serosa", "se...
## $ surg    <chr> "short", "short", "short", "long", "long", "short", "...
## $ node4   <chr> "yes", "no", "yes", "yes", "yes", "yes", "yes", "no",...
## $ time    <dbl> 968, 3087, 542, 245, 523, 904, 229, 3192, 3173, 3308,...
```

There are 888 subjects after tidying.

(c) Run the following code, replacing my initial data frame `d` with the name of your data frame read in the file. Describe the output that it produces. There are two things to note: what it shows for a quantitative variable and what it shows for a categorical variable

```
tidy.colon %>% mutate_if(is.character, factor) %>% summary()
```

```
##           id           rx           sex           age           obstruct
## Min.      : 1.0   Lev   :294   female:428   Min.      :18.00   no :717
## 1st Qu.:234.8   Lev+5FU:289   male  :460   1st Qu.:53.00   yes:171
## Median :466.5   Obs    :305                Median :61.00
## Mean    :466.5                Mean    :59.81
## 3rd Qu.:700.2                3rd Qu.:69.00
## Max.     :929.0                Max.     :85.00
## perfor    adhere          nodes          status          differ
## no :861    no :760   Min.      : 0.000   no recur:442   moderate:653
## yes: 27    yes:128   1st Qu.: 1.000   recur    :446   poor      :145
##                               Median : 2.000                well      : 90
##                               Mean    : 3.663
##                               3rd Qu.: 5.000
##                               Max.     :33.000
##           extent          surg          node4          time
## contiguous: 37   long :238   no :653   Min.      : 8.0
## muscle      :102   short:650   yes:235   1st Qu.: 379.8
## serosa      :730                Median :1556.0
## submucosa   : 19                Mean    :1410.4
##                               3rd Qu.:2289.8
##                               Max.     :3329.0
```

Let's interpret code correctly first; take our data frame, change any "character" variables to factors, and run the summary.

If we did not change the character variables, we would have something like this

```
summary(tidy.colon$rx)
```

```
##      Length      Class      Mode
##      888 character character
```

and it does not seem that anything can be interpreted.

However with the output above, it shows the frequencies of each categorical value of a categorical variable; this is helpful to answer questions like "Are there more males than females in the data set?".

As for quantitative variables, they stay the same when you regularly call `summary` on `tidy.colon`; displaying the minimum value, maximum value, interquartile range and the mean. For instance, the below provides exactly the same as the above:

```
summary(tidy.colon$id)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.0  234.8   466.5   466.5  700.2   929.0
```

(d) Obtain a suitable response variable for a Cox proportional-hazards model, using the time until recurrence and the column that says whether colon cancer recurred or not for each subject. Display the first few values of your response variable

This calls for `Surv` from the `survival` package. Thanks to Prof. Butler, the time variable is in the tidied data set.

```
y=with(tidy.colon, Surv(time, status=="recur"))
```

And as instructed, we display the first few values from the response variable

```
head(y)
```

```
## [1] 968 3087+ 542 245 523 904
```

(e) The second value (and also 8th, 9th, and 10th) of your response variable is displayed with a plus sign. What does that mean?

This means that the specific patient is **censored**. **Censored** follows the idea that there is not a definitive time measurement for a subject to have an event occur when data was recorded. That is, at the time the data was recorded, the patient did not have colon cancer recur, and it has been 3087 (and possibly more) days since colon cancer recurred (from what I am assuming is since chemotherapy).

We can somewhat confirm this with the following:

```
tidy.colon %>% select(status, time)
```

```
## # A tibble: 888 x 2
##   status    time
##   <chr>    <dbl>
## 1 recur     968
## 2 no recur 3087
## 3 recur     542
## 4 recur     245
## 5 recur     523
## 6 recur     904
## 7 recur     229
## 8 no recur 3192
## 9 no recur 3173
## 10 no recur 3308
## # ... with 878 more rows
```

As we can see, the second, eighth, ninth, and tenth row have a status of “no recur”.

(f) Fit a Cox propotional hazards model predicting the response variable you made in part (d) from all the other variables except for

i. those that were used in making the response variable

ii. those that make no sense to include in a prediction model. You don't need to display your results

(Hint: in a model, `.` on the right side of the squiggle means “all the variables in the data frame except for the one used on the left of the squiggle, if that was in the data frame”)

I still have `?colon` on the bottom right pane of RStudio; the obvious ones that should be ruled out are:

- time
- status

as for ii. `id` seems insignificant as it is just the patient's id. As for the others, the only one that does stand out is `node4`. `node4` just determines if the patient has more than 4 lymph (positive) nodes. However, there is already a variable called `nodes` which determines the number of lymph nodes the patient has, which makes `node4` to be a bloated variable that is derived from `nodes`. `etype` is already out since it was tidied.

All other variables seem to be okay; I personally had to look up what they mean, and it seems that perforation, adherence, differentiation, extent (of local spread), and time from surgery to registration (for chemotherapy) all seem relative. I find that age and sex are necessary with any medical record.

And so, here is the Cox proportional hazards model:

```
tidy.colon.1 = coxph(y~.-id - time - status - node4, data=tidy.colon)
# keep data = tidy.colon for later!
```

(g) Use `step` with `trace=0` to do a backward elimination of non-significant explanatory variables of the previous part, one at a time. Save the step into a variable, and display the `drop1` output of your final model. Are all the remaining variables significant at $\alpha = 0.10$?

Note: for the remainder of this question, use the output model from `step` as your “best model”. I don’t need you to eliminate any more variables

As instructed, we use `step`

```
tidy.colon.2 = step(tidy.colon.1, trace=0, test="Chisq")
```

then we display the `drop1` output

```
drop1(tidy.colon.2, test="Chisq")

## Single term deletions
##
## Model:
## y ~ rx + sex + obstruct + nodes + differ + extent + surg
##      Df    AIC    LRT  Pr(>Chi)
## <none>      5658.0
## rx        2 5675.3 21.361 2.299e-05 ***
## sex       1 5658.7  2.739 0.0979129 .
## obstruct  1 5658.5  2.519 0.1125116
## nodes     1 5707.4 51.451 7.342e-13 ***
## differ    2 5658.2  4.261 0.1188055
## extent    3 5670.2 18.195 0.0004009 ***
## surg      1 5661.0  5.085 0.0241313 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

After running `step`, I oddly see two variables that do not seem significant when $\alpha = 0.1$ which is `obstruct` and `differ` as their p-values are higher than α . Ordinarily, I would call for further backwards elimination, but that is not requested from the note.

(h) The major focus of the researchers was on whether the treatment affected the time to recurrence. Is treatment in your output model from step? Use your results from part (c) to obtain “representative” values for all the other explanatory variables in that model: the median for quantitative variables and the most frequently observed category for categorical variables

As the “best model” included the variable rx as its p-value is small, which stands for treatment. We can conclude that treatment affected the time to recurrence.

Now, as requested: the median for quantitative variables and the most frequently observed category. I somewhat see where this is going:

```
Sex = "male"
Obstruct = "no"
Nodes = 2.0
Differ = "moderate"
Extent = "serosa"
Surg = "short"
```

(i) Make a data frame containing each of the three treatments combined with the representative values you obtained in the previous part. This is going to be an input to a prediction, so make sure the columns of your new data frame have the same names as the variables they are values of

Sounds great, now we just create rx

```
rx = data.frame(rx=c("Obs", "Lev", "Lev+5FU"))
rx
```

```
##      rx
## 1    Obs
## 2    Lev
## 3 Lev+5FU
```

alright, now for crossing!

```
my.df = crossing(rx, sex=c(Sex), obstruct=c(Obstruct),
                 nodes=c(Nodes), differ=c(Differ),
                 extent=c(Extent), surg=c(Surg))
my.df
```

```
##      rx sex obstruct nodes  differ extent  surg
## 1    Obs male      no     2 moderate serosa short
## 2    Lev male      no     2 moderate serosa short
## 3 Lev+5FU male      no     2 moderate serosa short
```

(j) Obtain predicted survival curves for each of the three treatments and the representative values for the other variables you obtained earlier. Bear in mind that we will be plotting the results shortly

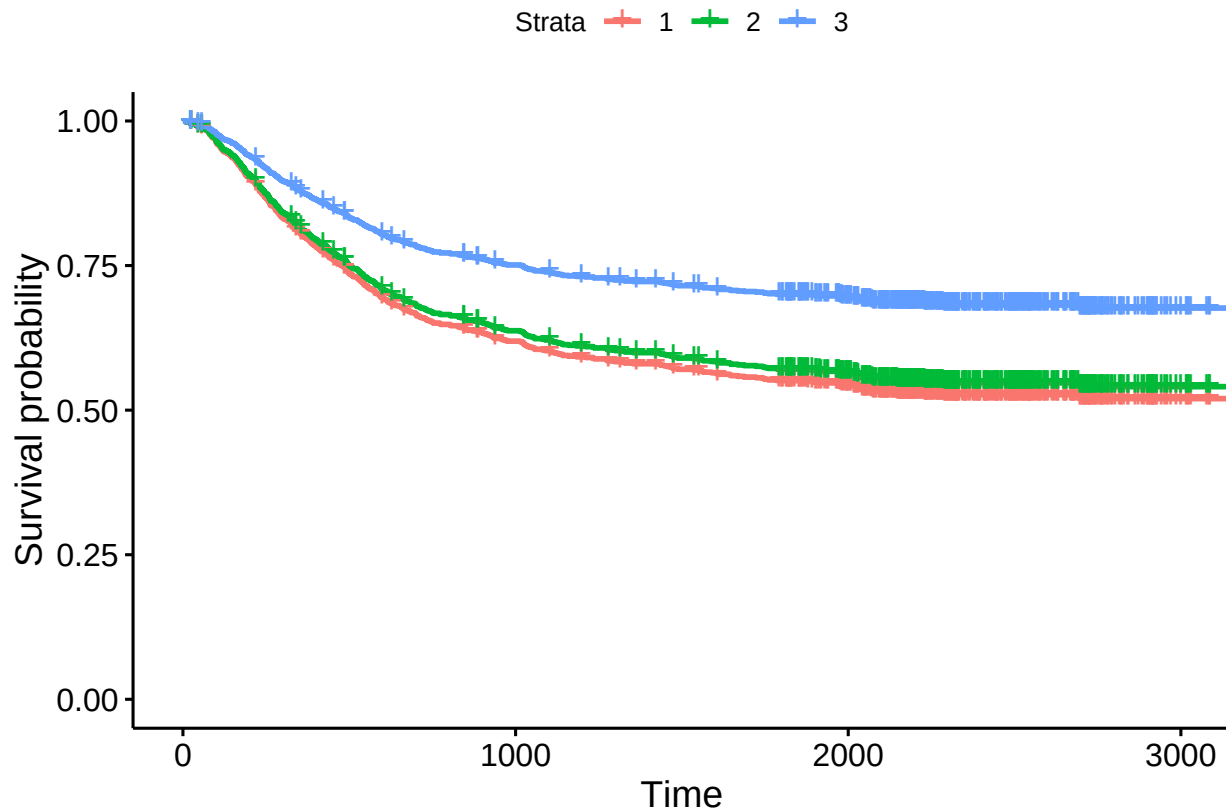
This should be using, we just use survfit

```
p = survfit(tidy.colon.2, my.df, data=tidy.colon)
# I would do summary on p, but the output for it is large, and would be inconvenient
```

(k) Make a plot of the predicted survival curves for the three different treatments, and describe briefly how the three treatments compare. In your description, you should use the full names for the treatments, as far as they are known.

We can use `ggsurvplot`

```
ggsurvplot(p, conf.int=F)
```



The first thing we have to note is that stratum 1 is an observational treatment (the orange line), stratum 2 is the levamisole treatment (the green line), and stratum 3 is the levamisole and flurouacil [5-FU] (the blue line).

In our case, the main question that we are answering with this graph is “which treatment would cause the least amount of recurrences of colon cancer, relative to time (days)?”

Usually “up and to the right” is the best possibility, and we find that stratum 3 is the best. That is, the levamisole and fluroacil treatment is much more effective than just levamisole or the observational treatments. Thus, we can say that the levamisole and fluroacil treatment is less likely for recurrences of colon cancer in comparison to the other two treatments.

Stratum 1 (observational treatment) and 2 (levamisole treatment) are relatively close, with the levamisole treatment being just a little bit better than the observational treatment. Thus, it is more likely that colon cancer recurrence may occur more in the observational treatment in comparison to the fluroacil treatment (non 5-FU).

(l) Display the summary of your best model. How is some of the output consistent with your graph?

```
summary(tidy.colon.2)
```

```
## Call:
## coxph(formula = y ~ rx + sex + obstruct + nodes + differ + extent +
##       surg, data = tidy.colon)
##
##      n= 888, number of events= 446
##
##              coef exp(coef) se(coef)      z Pr(>|z|)
## rxLev+5FU      -0.453521  0.635387  0.123940 -3.659 0.000253 ***
## rxObs          0.061564  1.063498  0.111267  0.553 0.580060
## sexmale       -0.158627  0.853315  0.095793 -1.656 0.097735 .
## obstructyes    0.190504  1.209859  0.117906  1.616 0.106153
## nodes          0.077968  1.081088  0.009376  8.316 < 2e-16 ***
## differpoor     0.266222  1.305025  0.125995  2.113 0.034605 *
## differwell     0.019696  1.019891  0.164385  0.120 0.904631
## extentmuscle   -0.957952  0.383678  0.269836 -3.550 0.000385 ***
## extentserosa   -0.436551  0.646262  0.212276 -2.057 0.039732 *
## extentsubmucosa -1.350264  0.259172  0.543101 -2.486 0.012911 *
## surgshort      -0.238965  0.787442  0.104288 -2.291 0.021941 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##              exp(coef) exp(-coef) lower .95 upper .95
## rxLev+5FU          0.6354      1.5738   0.49836   0.8101
## rxObs              1.0635      0.9403   0.85512   1.3227
## sexmale            0.8533      1.1719   0.70725   1.0296
## obstructyes        1.2099      0.8265   0.96023   1.5244
## nodes              1.0811      0.9250   1.06140   1.1011
## differpoor         1.3050      0.7663   1.01946   1.6706
## differwell         1.0199      0.9805   0.73897   1.4076
## extentmuscle        0.3837      2.6064   0.22609   0.6511
## extentserosa        0.6463      1.5474   0.42630   0.9797
## extentsubmucosa     0.2592      3.8584   0.08939   0.7514
## surgshort           0.7874      1.2699   0.64187   0.9660
##
## Concordance= 0.663 (se = 0.013 )
## Rsquare= 0.125 (max possible= 0.998 )
## Likelihood ratio test= 118.3 on 11 df,  p=<2e-16
## Wald test              = 136.8 on 11 df,  p=<2e-16
## Score (logrank) test = 141.8 on 11 df,  p=<2e-16
```

Recall in (d) that we set a suitable response variable, and the key thing in our output was “**status==recur**”; if this is true, the response value is a 1, otherwise the response value is 0.

With this knowledge in mind, a positive coefficient means that there will be a higher probability of a recurrence due to what we did in (d).

As we see in our summary, there is a negative coefficient for **rxLev+5FU** (at -0.453521), that is, Lev+5FU should be providing a decrease of a recurrence in colon cancer. If we look at our graph, we find that the Lev+5FU has the highest probability of “surviving”, or in our case, the highest probability a patient does not have colon cancer recurring. This seems consistent.

As for **rxObs**, we see that there is a positive coefficient at 0.061564. That is, an observational treatment should be increasing the probability of a recurrence in colon cancer. If we look at our graph, we find that the observational treatment has the lowest probability that a patient does not have colon cancer recurring. By exhaustion, we know that **rxLev** is in the “baseline” of our model, and it is expected to be in between **Lev+5FU** and **rxObs** (but much more closer to **rxObs** due to that small coefficient). Looking at our graph, just levamisole (non 5FU) is in between the levamisole 5FU treatment and the observational treatment, as expected.

Thus, our output seems consistent with the graph above.